Book Repository Layout

The Codynamic Book Machine keeps its working state in a repository that is intentionally split across a small number of durable surfaces. Rather than treating the manuscript as a single mutable file, the system separates outline structure, section source material, generated TEX payloads, reference data, runtime logs, build outputs, media requests, diagram memory, and artifact registries. That separation is not incidental; it is what allows the book to be inspected, revised, rebuilt, and audited without collapsing authorial intent into a single opaque artifact.

At the top of the repository sits the outline and work definition. This is the canonical description of the book: its title, metadata, recursive structure, dependencies, and section inventory. The outline is the source of truth for what exists in the manuscript and how the manuscript is organized. Each chapter and section is represented as a structured node with an identifier, a number, a title, and a content file path. In practice, this means that the repository can answer two distinct questions at once: what the book is about, and where the editable payload for each part lives.

Section source files form the next layer. These files preserve imported notes, normalized source text, or other upstream material that informed a section before it was rendered into final prose. They are useful as a trace of provenance and as a staging area for conversion work. A section agent reads the source material, the outline intent, and the surrounding context, then writes the generated TEX body to the section payload file for that section. The payload is the authored output, while the source file remains the record of what was imported or supplied.

The generated TEX payloads are stored separately from the source material so that the manuscript can be assembled from many independently produced sections. This makes local revision possible without rewriting the whole book. A section payload should contain only the body content for that section, not the document preamble or wrapper commands. The assembly step later combines these payloads into the compiled manuscript. Keeping payloads isolated also makes it easier to compare drafts, attribute changes, and repair a single section without disturbing the rest of the document.

References are managed as a shared repository resource rather than as ad hoc citations embedded in arbitrary drafts. The bibliography file is the canonical store for registered citation keys, and section agents are expected to use only keys that have already been registered. When a section needs support that is not yet available, the section agent must request research or reference registration instead of inventing a citation. This keeps the manuscript honest about its evidentiary basis and makes citation support a coordinated task rather than a hidden authoring shortcut.

Logs provide the runtime memory of the system. They record task execution, agent messages, callbacks, and other durable traces of coordination. Because the system is message-driven, the logs are not merely diagnostic; they are part of the manuscript's operational history. A reader or maintainer can inspect them to understand why a section changed, which agent acted, what prompt or callback was involved, and how a revision was triggered. In that sense, the logs are a provenance layer for the authoring process itself.

Build artifacts capture the compiled state of the manuscript. These include generated TEX outputs, PDFs, and any intermediate files produced during assembly or repair. They are distinct from the section payloads because they represent the result of combining many source surfaces into a single deliverable. When compilation fails, the build artifacts and logs together provide the evidence needed to repair the relevant section or adjust the assembly process. The compiled outputs are therefore not just publication products; they are also feedback signals for the authoring system.

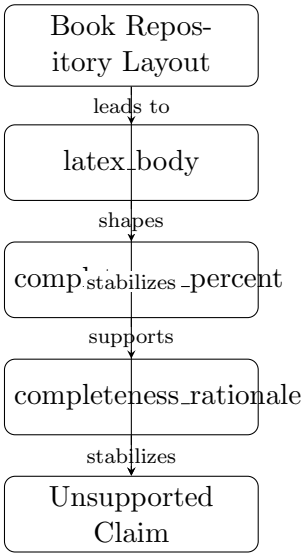
Media requests and diagram memory extend the repository beyond text. When a section would benefit from a visual, the section agent can request a diagram rather than fabricating one inline.

The request records the section context, the purpose of the figure, and the conceptual structure that the diagram should express. The diagram agent then creates the asset and returns a callback with a path and insertion hint. Diagram memory preserves the rationale for each generated figure, including its purpose, layout, nodes, edges, and distinctness from prior diagrams. This prevents the repository from accumulating redundant visuals and makes diagram generation a managed part of the manuscript's state.

Artifact registries tie these surfaces together. They provide a durable index of what has been created, where it lives, and how it should be used. A registry can point from a section identifier to its payload file, from a diagram request to its generated asset, or from a build to its output files and logs. In effect, the registry is the book's map: it lets agents and maintainers navigate the repository without guessing where a given artifact belongs.

Taken together, these repository layers make the book machine inspectable. Outline files define structure, source sections preserve upstream material, TEX payloads hold generated prose, references support claims, logs preserve runtime history, build artifacts show compiled results, media requests and diagram memory manage visuals, and registries connect the whole system. The repository layout is therefore not just a file organization scheme; it is the operational boundary that keeps the manuscript coherent while many agents work on it at once.

Figure 1 summarizes the repository as a set of coordinated surfaces rather than a single monolithic manuscript file. The outline anchors structure, section sources feed drafting, payloads store authored prose, and the registry layer links the resulting artifacts back to their origins.



Conceptual diagram for 'Book Repository Layout': Describe outline files, source sections, TeX payloads, references, logs, build artifacts, media requests, diagram memory, and artifact registries.

Figure 1: Conceptual diagram for Book Repository Layout: outline files, source sections, TEX payloads, references, logs, build artifacts, media requests, diagram memory, and artifact registries.